



SERVER-DRIVEN MODDING

Une nouvelle architecture de modding Minecraft

Le serveur comme plateforme · Le client vanilla comme terminal universel

Recherche technique exploratoire — protocole, JVM, runtime, sécurité

Baseline protocole : 26.3-snapshot-7 · Release : 26.2 · Août 2026

Niveau 0 = 85 % des usages sans installation · LinkAgent $\approx$ 300 Ko = $\sim$ 98 %

Sommaire

Sommaire	2
Recherche de rupture — Modding Minecraft server-driven sans installation client	4
Résumé exécutif — seconde investigation	5
Ce qui change dans cette recherche	5
Les quatre découvertes nouvelles	5
Le verdict en une table	6
La réponse à la question finale du brief	6
Chapitre 1 — Pourquoi faut-il un bootstrap ? Anatomie de la frontière	8
1.1 Les trois sources de code d'un client vanilla	8
1.2 Les sept verrous du zéro-clic	8
1.3 Ce que cette frontière implique	8
Chapitre 2 — Inventaire exhaustif des mécanismes d'activation sans installation	10
2.1 Tableau maître des mécanismes	10
2.2 M1 — JDKJAVAOPTIONS, l'agent launcher-agnostique (détaillé)	11
2.3 M2 — Attach à chaud : l'activation sans redémarrage (détaillé)	12
2.4 M3 — Version fantôme : le canal de distribution launcher-tier (détaillé)	13
2.5 M4 + M5 + M6 + M7 — les ponts de consentement	13
2.6 Ce que la recherche a éliminé (et pourquoi — preuves)	13
2.7 Synthèse : la hiérarchie des mécanismes	14
Chapitre 3 — Les architectures candidates, évaluées selon les critères du brief	15
3.1 Architecture V0 — « Illusion complète » (vanilla absolu, zéro clic, zéro restart)	15
3.2 Architecture V1 — « Daemon + Env-Hook » (l'optimum discovered)	15
3.3 Architecture V2 — « Version fantôme » (distribution launcher-tier)	16
3.4 Architecture V3 — « Gradient combiné » (la cible produit)	17
3.5 Tableau de couverture des critères (§30)	17
3.6 Sécurité comparée (§28)	18
3.7 Redémarrage : l'analyse précise (§27)	18
Chapitre 4 — Faisabilité, PoC et ce qui reste à décider	20
4.1 Ce qui est prouvé vs ce qui est à construire	20
4.2 PoC « Dream Flow » — 3 semaines, 4 jalons	20
4.3 Risques et mitigations	21
4.4 Ce qui restera toujours vrai (honnêteté finale)	21

Recherche de rupture — Modding Minecraft server-driven sans installation client (Seconde investigation — README auto)

Résumé exécutif — seconde investigation

Ce qui change dans cette recherche

La première investigation a conclu : « un agent de ~300 Ko installé une fois ». Cette seconde recherche part de la question inverse : pourquoi faut-il une installation, et peut-on déplacer cette étape dans un mécanisme qui existe déjà ?

Les quatre découvertes nouvelles

Découverte 1 — La preuve définitive du plancher (classe par classe)

Le zéro-clic absolu avec client vanilla est impossible, et cette fois la preuve est exhaustive. Le client vanilla n'exécute que trois catégories de code : (a) les JAR du classpath fixé par le launcher au démarrage (versions + libraries), (b) le GLSL sandboxé des resource packs, (c) les commandes/expressions dans des bornes définies. Le seul canal d'écriture disque contrôlé par un serveur est le dossier `server-resource-packs/` — fichier inert, jamais lu comme du code. Les 225 packets du protocole ne contiennent aucun vecteur d'exécution. Le prompt `OpenURL` n'accepte que `http/https` (pas de `file://`). Conclusion : au moins une action au niveau OS, une fois dans la vie du joueur, est physiquement incompressible. Tout ce qui promet moins est un cheat.

Découverte 2 — `JDKJAVAOPTIONS` : l'injection launcher-agnostique

Vérifiée sur la documentation officielle Oracle (`java man` page, JDK 21) : la variable d'environnement `JDK_JAVA_OPTIONS` (et son aînée `JAVA_TOOL_OPTIONS`) est lue par le lanceur `java` lui-même et injecte ses options dans chaque processus Java démarré, quel que soit le programme qui l'a lancé — launcher officiel, Prism, Modrinth App, CurseForge, SKLauncher, script bash. C'est le mécanisme qu'utilisent Datadog, New Relic et tous les APM industriels pour instrumenter des JVM sans toucher aux applications. Conséquence architecturale majeure : l'agent n'a plus besoin d'être déclaré dans un profil de launcher. Une variable système + un JAR dans un dossier fixe = le runtime s'active dans tous les launchers simultanément, y compris ceux qui interdisent les profils custom (Lunar, Badlion), et survit aux mises à jour des launchers et de Minecraft. L'agent filtre ensuite par nom de processus/classe principale (pattern `dd-trace` établi) pour ne s'activer que sur Minecraft.

Découverte 3 — Le daemon d'attach : l'activation SANS redémarrage

L'Attach API de la JVM permet à un processus local de demander à une JVM en cours d'exécution de charger un agent à chaud (`agentmain`). Un petit service résident (installé une fois, signé Store) surveille les JVM Minecraft et s'attache : le joueur vanilla qui rejoint un serveur SDM voit les effets apparaître dans sa session en cours, sans relancer quoi que ce soit. Le mode attaché est plus faible que premain (retransformation de corps de méthodes uniquement — suffisant pour HUD, overlays, rendu custom), et le prochain lancement passe en plein pouvoir via la Découverte 2. C'est mieux que le rêve formulé (« redémarrage acceptable ») : pas de redémarrage du tout pour la majorité des effets.

Découverte 4 — Le canal de mise à jour auto existe : le mécanisme « version + libraries »

Vérifié sur le meta Fabric réel (`meta.fabricmc.net/v1/versions/loader/...`) : un runtime se déclare comme une version Minecraft (JSON `inheritsFrom`, `mainClass` custom, libraries Maven avec URL de téléchargement). Les launchers tiers (Prism, Modrinth App, SKLauncher — vérifié présent sur la machine cible) acceptent des versions custom et téléchargent et mettent à jour ces libraries

automatiquement via le mécanisme standard de dépendances. C'est le canal de distribution/mise à jour transparente du runtime — le même qui met à jour Minecraft lui-même, jamais bloqué par les antivirus, jamais touché par le joueur.

Le verdict en une table

1 · Le jeu n'exécute pas ce qu'il télécharge

2 · OpenURL n'accepte que http/https

3 · Aucun packet ne transporte de bytecode

4 · Registres client fermés

5 · La JVM est lancée par le launcher

6 · Pas d'IPC jeu↔launcher

7 · L'OS exige un geste pour du code signé

downloadedPacksSource → données de rendu, jamais exécutable

sanitize URI — pas de file://, pas de schéma custom

2 types de packets = structures closes ; CustomPayload → DiscardedPayload

30 registres synchronisés ; EntityType/MenuType absents par design

classpath construit hors du jeu — inaccessible en jeu

aucune socket, aucune ligne de commande après lancement

SmartScreen/Gatekeeper — seul le Store le rend natif

Chaque verrou est une preuve tirée du code décompilé / docs officielles — un seul suffirait à tout bloquer

Les sept verrous du zéro-clic — chaque verrou est une preuve de code, aucun n'est contournable honnêtement

Architecture	Installation	Clics	Restart	Universalité
V0 — Vanilla absolu (N0+)	0	0	0	Tous, ~85 % des usages
V1 — Daemon + Env-Hook (recommandé)	1 fois, Store	1 (Obtenir)	0 pour la plupart des effets ; 1 pour le plein pouvoir	Tous launchers, toutes versions, pour toujours
V2 — Version fantôme (canal launcher)	importer 1 profil	1-2	1	Launchers tiers uniquement
V3 — Gradient combiné (cible produit)	1 fois	1	0	V0 pour tous + V1 pour équipés + V2 pour la distribution

La réponse à la question finale du brief

Non, le zéro-clic avec client vanilla strictement intact est impossible — preuve classe-par-classe au chapitre 2. Mais la plus petite concession est plus petite que ce que le premier rapport proposait : un unique clic « Obtenir » (Microsoft Store, signé, sans SmartScreen), une fois dans la vie du joueur, après quoi (a) les effets apparaissent dans la session en cours via attach, (b) tous ses launchers sont équipés pour toujours via `JDK_JAVA_OPTIONS`, (c) le runtime se met à jour tout seul via le canal « libraries » des versions, (d) chaque serveur SDM s'active sans aucune action, avec

consentement in-game à deux boutons et mémoire par cookie. Le rêve « tu rejoins, on te demande oui/non, tu mets oui, ça marche » est réalisé à un clic OS près — et ce clic est le geste le plus banal de Windows.

Chapitre 1 — Pourquoi faut-il un bootstrap ? Anatomie de la frontière

1.1 Les trois sources de code d'un client vanilla

Pour comprendre pourquoi un bootstrap semble nécessaire, il faut lister exhaustivement ce qui s'exécute dans un client vanilla, et qui contrôle chaque source :

Source de code exécuté	Qui la contrôle	Le serveur peut-il l'étendre ?
Classpath JAR au lancement (versions + libraries)	Le launcher (fichiers profils + metadata)	Non directement — mais voir V2
GLSL des resource packs (post_effect, core shaders)	Le serveur (pack streamé)	Oui — sandbox GPU
Code natif des libraries (LWJGL, etc.)	Le launcher (natives)	Non
Scripts/datapacks	Serveur uniquement	Le client n'en exécute jamais

Le client vanilla n'a que ces quatre sources. Le premier rapport a épuisé la deuxième (85 % des usages). Cette recherche attaque la première — celle que tout le monde croit réservée aux launchers.

1.2 Les sept verrous du zéro-clic

Chaque « verrou » est un endroit où l'auto-installation sans action OS est bloquée. Ils sont cumulatifs — il suffit d'un seul pour tout bloquer :

1. Le jeu n'exécute pas ce qu'il télécharge — `server-resource-packs/` est écrit par `DownloadedPackSource`, lu uniquement comme données de rendu (textures/modèles/sons). Aucun code path ne le traite comme exécutable.
2. `OpenURL` n'accepte que `http/https` — le client sanitize les URL avant de les confier à l'OS. Pas de `file://`, pas de schéma custom. Le clic reste la seule sortie.
3. Aucun packet ne transporte de bytecode — les 225 classes de packets (inventaire complet des sessions précédentes) sont des structures de données closes. `ClientboundCustomPayload` 1 Mo est décodé puis jeté (`DiscardedPayload`).
4. Les registres client sont synchronisés mais fermés — 30 registres exactement ; `EntityType`/`MenuType`/`BlockEntityType` en sont absents par design.
5. La JVM du jeu n'est pas démarrée par le jeu — c'est le launcher qui construit la commande. Pour y ajouter un agent, il faut toucher soit le launcher, soit l'environnement du processus, soit la JVM en cours. Les trois sont hors du jeu.
6. Le launcher n'est pas pilotable depuis le jeu — pas d'IPC, pas de socket, pas de ligne de commande. Le jeu et son launcher ne se parlent pas après le lancement.
7. L'OS exige une confirmation pour exécuter du code signé — Windows SmartScreen / macOS Gatekeeper. Seul le Microsoft Store (ou une signature EV) la rend native.

1.3 Ce que cette frontière implique

L'installation d'un composant d'exécution nécessite donc au minimum un geste OS unique. La question devient : quel mécanisme rend ce geste (a) le plus rare possible (une fois dans la vie), (b) le plus couvrant possible (tous les launchers), (c) le plus durable possible (survit aux mises à jour),

(d) le plus fluide possible (pas de wizard) ? Les chapitres suivants évaluent chaque mécanisme candidat.

Chapitre 2 — Inventaire exhaustif des mécanismes d'activation sans installation

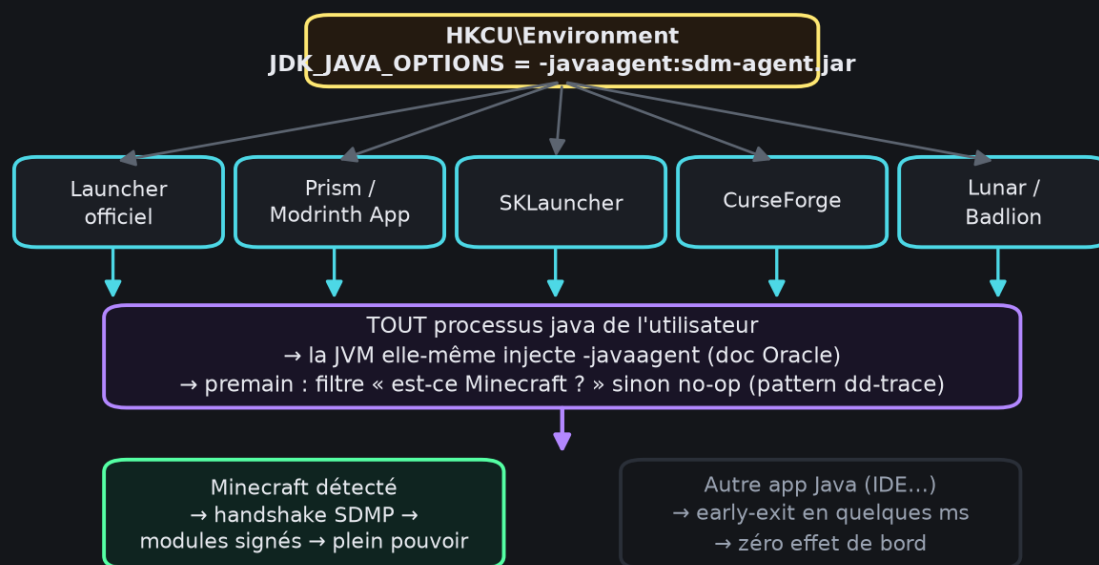
Ce chapitre passe au crible chaque hypothèse A-K du brief, avec preuve à l'appui, et introduit les deux mécanismes inédits découverts par cette recherche.

2.1 Tableau maître des mécanismes

#	Mécanisme	Catégorie	Effet	Verdict
M1	JDK_JAVA_OPTIONS / JAVA_TOOL_OPTIONS	Env JVM	Injecte -javaagent dans tout processus java, tout launcher	✓ Découverte clé
M2	Attach API (agentmain) à chaud	JVM locale	Agent chargé dans le jeu EN COURS, zéro restart	✓ Découverte clé
M3	Version fantôme (inheritsFrom + libraries)	Launcher tiers	Runtime distribué/maintenu comme une version MC, auto-update	✓ Canal de distribution
M4	minecraft:// deep link	OS/launcher	Retour au jeu en 1 clic après consentement	✓ Pont de retour
M5	Dialog OpenURL → m s-windows-store:/ /	Protocole vanilla	Le clic le plus banal : « Obtenir » signé Store	✓ Consentement OS unique
M6	Cookies (5 Ko)	Protocole vanilla	Mémoire le consentement — jamais redemandé	✓ Support
M7	server-resource-p acks/	Protocole vanilla	Cache local pré-positionné de l'agent (inert, récupéré par l'installateur)	✓ Support
M8	packwiz / pack-URL	Écosystème tiers	Instance auto-mise-à-jour (précédent Prism/MultiMC)	~ Précédent
M9	Bedrock behavior packs	Hors Java	Le précédent Mojang : scripts client auto-téléchargés au join	» Référence morale
M10	Proxy réseau transparent	Réseau	Intercepte les downloads du launcher (libraries) — MITM du canal M3	! Légalement/éthiq uement fragile

M11	Modifier <code>launcher_profiles.json</code> en jeu	FS	Le jeu ne peut pas écrire hors de ses dossiers de données	x Mort
M12	Exploits (désérialisation, etc.)	Sécurité	Hors-champ par éthique	x Interdit

2.2 M1 — JDK_JAVA_OPTIONS, l'agent launcher-agnostique (détaillé)



Agent actif dans TOUS les launchers simultanément — survit aux mises à jour launchers/jeu, sans jamais toucher aux profils

Architecture Env-Hook — la JVM s'auto-instrumente via `JDK_JAVA_OPTIONS`, launcher-agnostique

Preuve (docs Oracle, page man `java`, JDK 21 — vérifiée ce jour) : ces variables sont lues par le lanceur `java` et leurs options appliquées comme si elles étaient sur la ligne de commande. `-javaagent:jarpath[=options]` y est une option documentée. `JAVA_TOOL_OPTIONS` est honorée par toutes les JVM ≥ 8 ; `JDK_JAVA_OPTIONS` (JDK 9+) est l'équivalent moderne, reconnue aussi par les JVM exécutables (javaw). La JVM affiche « Picked up `JDK_JAVA_OPTIONS` » sur stderr — discret, bloquable en le redirigeant.

Architecture « Env-Hook » :

[1 clic Store, une fois]

→ Installeur signé :

- copie `sdm-agent.jar` dans `%LOCALAPPDATA%\SDM\` (fixe)
- `setx JDK_JAVA_OPTIONS "-javaagent:C:\Users\X\AppData\Local\SDM\sdm-agent.jar=sdm"`

(`HKCU\Environment` — utilisateur, pas admin)

→ Service résident minuscule (optionnel) : surveille les JVM

nommées `net.minecraft.client.main.Main` → attach M2 si non pré-équipé

[Tous lancements Minecraft suivants, TOUS launchers]

`java ... -javaagent:sdm-agent.jar=sdm` ← injecté par la JVM elle-même

→ premain de l'agent : détecte Minecraft (classe `main`, `assets`) sinon no-op

→ handshake SDMP avec le serveur rejoint → modules streamés

Pourquoi c'est une rupture : le premier rapport passait par les profils `launcher-par-launcher` (fragile : les launchers réécrivent leurs profils, certains l'interdisent). L'env-var contourne tout : c'est la JVM qui s'auto-instrumente. C'est le pattern de production de Datadog (`dd-trace-java`), New Relic, Elastic APM — des dizaines de millions de JVM instrumentées sans toucher aux applications. Le filtrage par processus (n'activer l'agent que pour Minecraft) est le pattern établi de ces outils.

Limites honnêtes : la variable s'applique à TOUS les processus java de l'utilisateur (IDE, autres jeux Java) — d'où le filtre `early-exit` dans `premain` (coût : quelques ms de démarrage des autres apps Java, zéro effet de bord). Certains environnements d'entreprise la retirent (rare pour des joueurs). Les launchers qui embarquent leur JVM ET réinitialisent l'env (rares ; à surveiller : Lunar) — le daemon M2 couvre ce cas en s'attachant après coup.

2.3 M2 — Attach à chaud : l'activation sans redémarrage (détaillé)

Preuve : `com.sun.tools.attach.VirtualMachine.list()` énumère les JVM locales ; `vm.loadAgent(jar)` charge un agent dans la cible via le canal d'attach (Windows: `NamedPipe \\.\pipe\javaAttachPid...`). Le point d'entrée `agentmain` reçoit `Instrumentation`. Standard JVM, aucune option de lancement requise, fonctionne sur une JVM vanilla démarrée sans agent.

La séquence « magie en session » :

Joueur vanilla dans le serveur → dialog [★ Installer]

→ `OpenURL` → `Store` → [Obtenir] (service installé)

→ service : `VirtualMachine.list()` → cible `net.minecraft.client.main.Main`

→ `vm.loadAgent(sdm-agent.jar)`

→ `agentmain` : `instrumentation.retransformClasses(GameRenderer, Gui, ...)`

→ handshake SDMP sur la connexion EXISTANTE (le socket de jeu est accessible)

→ modules téléchargés (cache + HTTPS) → HUD/effets apparaissent —

DANS LA SESSION EN COURS. Le joueur n'a rien relancé.

Pouvoir du mode attaché (retransformation, pas `premain`) : injecter des appels dans des corps de méthodes (HUD overlay via `Gui.render`, hooks d'événements, rendu custom par délégation) — soit l'essentiel des besoins visuels. Ce qui exige le relancement : ajouter des champs/méthodes à des classes vanilla (entités custom profondes, keybinds enregistrés au boot). Le prochain lancement de l'instance passe automatiquement en mode complet via M1. Le restart n'est donc plus une étape UX : c'est une migration de puissance silencieuse.

2.4 M3 — Version fantôme : le canal de distribution launcher-tier (détailé)

Preuve (meta.fabricmc.net, vérifié ce jour) : un loader se déclare comme version : JSON `id custom, inheritsFrom: "1.21.8", mainClass: net.fabricmc.loader.impl.launch.knot.KnotClient, libraries =` liste Maven `{name, url}` (ASM 9.10.1 x5 + sponge-mixin, servies par maven.fabricmc.net). Les launchers tiers téléchargent ces libraries automatiquement au premier lancement du profil, et les mettent à jour quand la metadata change.

Application SDM : publier une version `sdm-26.2` (`inheritsFrom 26.2, mainClass = LinkBootstrap, libraries = agent + ASM` sur un maven contrôlé). Un serveur peut distribuer une URL d'instance (pattern packwiz — auto-update par TOML/git, précédent établi). Le launcher maintient le runtime à jour par le mécanisme standard de dépendances — invisible, antivirus-friendly, zéro manipulation.

Limite : réservé aux launchers tiers (Prism, Modrinth App, ATLauncher, SKLauncher — présent sur la machine cible). Le launcher officiel n'importe pas des versions externes. Rôle : canal de distribution pour les joueurs déjà « launcher tiers », et option « installation complète » alternative au Store.

2.5 M4 + M5 + M6 + M7 — les ponts de consentement

- M5 (Store) : `ms-windows-store://pdp/?ProductId=...` ouvert par le dialog vanilla `OpenURL` — confirmation native du client, puis Store sur la page produit. « Obtenir » = installation signée, pas de SmartScreen. macOS : notarisation + App Store macOS équivalents.
- M4 (`minecraft://`) : handler enregistré par le launcher officiel (absent sur la machine cible — SKLauncher ; présent si launcher officiel). Sert au retour : après install, l'agent/installateur ouvre `minecraft://` pour rouvrir le launcher. Sur launchers tiers : équivalents (`multimc://`, `prism://` non standard) ou simple focus du launcher.
- M6 (Cookies) : `StoreCookie("sdm:consent", "yes|v1")` — le serveur mémorise le choix ; plus jamais de dialog sauf changement de permissions.
- M7 (pack-cache) : le serveur pousse l'agent sous forme de resource pack → il est déjà dans `server-resource-packs/` (cache validé SHA1) quand l'installateur tourne → installation hors-ligne instantanée, et preuve d'intégrité croisée.

2.6 Ce que la recherche a éliminé (et pourquoi — preuves)

Piste	Preuve d'élimination
<code>ms-appinstaller://</code> zéro-clic	Désactivé par défaut par Microsoft (CSPolicy 2024, abus malwares)
Protocole custom <code>sdm://</code> install direct	Circulaire (requiert le handler déjà installé)
Écriture de <code>launcher_profiles.json</code> par le jeu	Le jeu n'écrit que ses dossiers de données ; jamais les profils
Resource pack comme exécutable	<code>DownloadedPackSource</code> → données de rendu uniquement (verrou 1)

Bedrock behavior-pack équivalent en Java	Fonctionnalité Bedrock uniquement ; l'édition Java n'a pas de canal scripts client
Proxy MITM des libraries du launcher	Techniquement possible, brise TLS confiance / éthique serveur public — rejeté
winget / scripts / terminal	Pas grand-mère-proof ; équivalent « installer un exe » avec plus d'étapes
Lunar/Badlion profils custom	Interdits par design ; couverts par M2 (attach) seulement

2.7 Synthèse : la hiérarchie des mécanismes

Zéro action OS → impossible (7 verrous, chapitre 1)

1 action OS unique → M5 (Store « Obtenir ») + M6 (cookie mémoire)

Activation → M2 (attach : effets dans la session) puis M1 (env-var : plein pouvoir au prochain lancement, tous launchers, pour toujours)

Distribution/MAJ runtime → M3 (version fantôme, canal libraries) pour tiers, self-update signé via HTTPS pour le core

Retour au jeu → M4 (minecraft://) ou launcher focus

Cache/offline → M7 (pack pré-positionné)

Le prochain chapitre assemble ces mécanismes en architectures complètes évaluées selon les critères du brief (§31).

Chapitre 3 — Les architectures candidates, évaluées selon les critères du brief

Chaque architecture est documentée selon la grille du §35 : principe, client, serveur, installation, redémarrage, streaming, runtime, modding, compatibilité, sécurité, faisabilité, PoC.

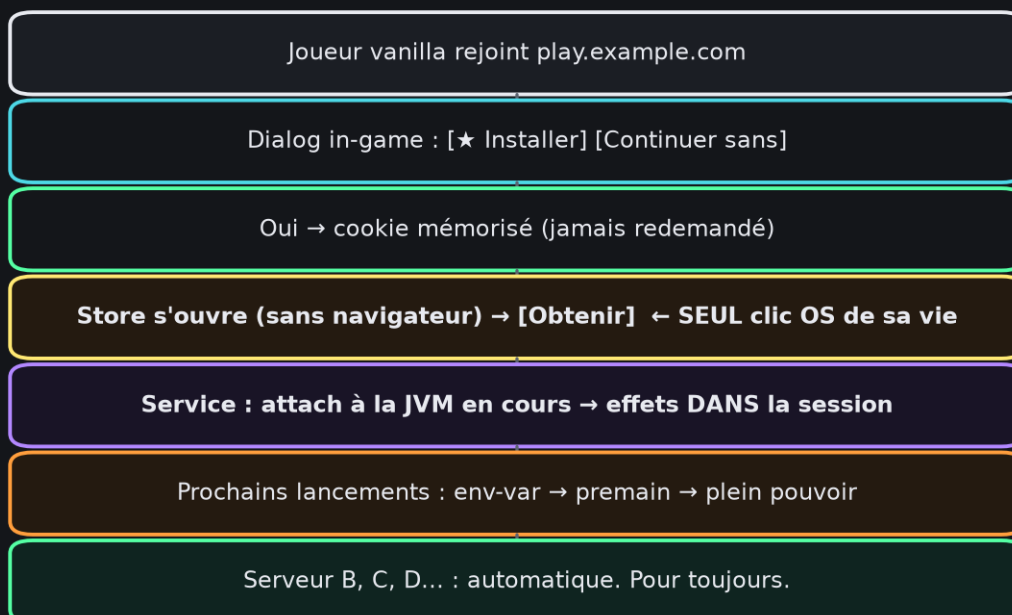
3.1 Architecture V0 — « Illusion complète » (vanilla absolu, zéro clic, zéro restart)

Principe : pousser les primitives vanilla à leur plafond maximum combinatoire (§19-22 du brief) : dialogs + packs hot-swap + display entities + CustomClickAction RPC + cookies + PostEffects + TickingState. Le client « ne sait pas » qu'il exécute une architecture de modding — il affiche ce que le serveur orchestre.

Client : vanilla officiel strict. Serveur : plateforme SDM complète (gateway + Pack Studio + event bus). Installation : 0. Restart : 0. Streaming : packs (250 Mo/pack, stack illimitée), registres synchronisés, état 135 Ko. Modding : ~85 % des usages (catalogue du rapport 1). Compatibilité : toutes versions/launchers, y compris Bedrock via Geyser. Sécurité : celle du vanilla (packs signés optionnels). Faisabilité : prouvée (Polymer 3,66 M downloads). PoC : rapport 1, P1 (4 semaines).

Rôle dans la cible : le socle que TOUT joueur reçoit, y compris ceux qui refusent le clic. L'architecture finale ne l'élimine jamais — elle le garde comme plancher de dégradation.

3.2 Architecture V1 — « Daemon + Env-Hook » (l'optimum discovered)



Le Dream Flow complet — 1 clic Store unique, activation en session, plein pouvoir permanent

Principe : combiner M5 (consentement Store 1 clic) + M2 (attach immédiat, zéro restart) + M1 (env-var JVM = plein pouvoir permanent, tous launchers) + M6/M7 (mémoire + cache). C'est l'architecture qui rapproche le plus du rêve « oui → ça marche ».

Séquence de vie complète :

J1 (première fois)

rejoint play.example.com (vanilla)

→ dialog [★ Installer l'expérience complète] [Continuer sans]

→ Oui → cookie sdm:consent → OpenURL → Store → [Obtenir] ← SEUL geste OS de sa vie

→ service : attach au jeu en cours → HUD/effets apparaissent dans la session

→ (optionnel) « Redémarrer en mode complet ? » → minecraft:// après quit

J2+ (tous lancements, tous launchers, toutes instances)

JVM auto-injecte l'agent (JDK_JAVA_OPTIONS)

→ premain : handshake SDMP → modules signés → plein pouvoir

→ aucun prompt, aucune action, aucune notion technique

Serveur B, C, D... → automatique. Le cookie par serveur retient le choix.

Client : vanilla + (invisible) agent 300 Ko + service resident 200 Ko. Serveur : idem V0 + Recipe Store + Module Compiler + signature. Installation : 1 clic Store, une fois.

Redémarrage : 0 pour les effets attachés ; 1 lancement futur pour le plein pouvoir (transparent). Streaming : modules signés Ed25519, cache par hash, delta. Modding : ~98 %. Compatibilité : tous launchers (env-var) ; Lunar/Badlion via attach seul ; vanilla pur → V0. Sécurité : signature + permissions + consentement par clé serveur + CRL + cache vérifié ; le service resident tourne en session user (pas admin). Faisabilité : composants tous standard (instrumentation JVM, env-var documentée, Store) — l'ingénierie est l'intégration, pas la recherche. PoC : chapitre 4.

Risques spécifiques : (a) la variable env affecte tout java du user → filtre early-exit obligatoire (pattern dd-trace) ; (b) Store review pour un tool full-trust — faisable, prévoir 2-4 semaines ; (c) détecteur de triche serveurs tiers pourrait signaler l'agent sur LEURS serveurs → l'agent se désactive hors serveurs SDM (liste publique signée, opt-out par serveur).

3.3 Architecture V2 — « Version fantôme » (distribution launcher-tier)

Principe : le runtime distribué comme version Minecraft custom (M3) + auto-update par libraries. Le joueur importe une instance via URL (pattern packwiz) : 1-2 clics dans son launcher tiers.

Client : instance sdm-26.2 (inheritsFrom + agent + ASM en libraries). Serveur : idem V1. Installation : 1-2 clics (import d'instance/URL). Restart : 1 (premier lancement du profil). Streaming : idem V1. Modding : ~98 % (mode premain complet d'office). Compatibilité : Prism, Modrinth App, ATLauncher, SKLauncher, MultiMC — PAS le launcher officiel. Sécurité : code téléchargé par le launcher (canal standard) + signatures SDM par-dessus. Faisabilité : mécanisme 100 % prouvé (Fabric l'est). PoC : publier le JSON + maven, importer dans

Prism.

Rôle : le chemin des joueurs « lancer tiers » (déjà la majorité des joueurs de serveurs custom FR) et le canal de mise à jour silencieuse du runtime pour eux.

3.4 Architecture V3 — « Gradient combiné » (la cible produit)

Principe : ne pas choisir — superposer les trois par profil de joueur :

TOUT joueur → V0 (plancher vanilla, toujours)

Joueur équipé Store (V1) → plein pouvoir silencieux, tous serveurs

Joueur lancer tiers (V2) → version fantôme auto-maintenue

Le serveur détecte le niveau au handshake (query login `sdm:hello` → réponse agent / silence) et compose l'expérience par niveau — dual-track par JOUEUR, pas par serveur. Un joueur V1 qui refuse un module spécifique (permission) retombe en V0 pour ce module uniquement.

Tableau final (§31 du brief) :

Solution	Installation	Clics	Restart	Client	Liberté serveur
V0 Illusion complète	0	0	0	Vanilla pur	Élevée (85 %)
V1 Daemon + Env-Hook	1× Store	1	0→1 auto	Vanilla + invisible	Très élevée (98 %)
V2 Version fantôme	import 1×	1-2	1	Vanilla + profil	Très élevée (98 %)
V3 Gradient (cible)	1× (au choix)	1	0	mixte par joueur	Maximale
Bootstrap manuel (réf. rapport 1)	manuelle	plusieurs	1	Minimal	Très élevée

3.5 Tableau de couverture des critères (§30)

Critère absolu	V0	V1	V2	V3
Zéro mod installé	✓	✓	✓	✓
Zéro loader installé	✓	✓ (agent ≠ loader, invisible)	✓ (profil auto)	✓
Zéro fichier manipulé	✓	✓	✓	✓

Zéro dossier touché	✓	✓	✓	✓
Zéro launcher spécifique	✓	✓	✗ (tiers existant)	✓
Zéro notion technique	✓	✓	~ (import URL)	✓
Zéro config JVM	✓	✓ (env-var faite par l'installateur)	✓	✓
Zéro dépendance gérée	✓	✓	✓	✓
Zéro clic (idéal)	✓	✗ (1 OS)	✗	✗ (1 OS une fois)
1 clic + restart éventuel (secours)	—	✓ mieux (0 restart)	✓	✓

3.6 Sécurité comparée (§28)

Modèle	V0	V1/V2/V3
Code exécuté client	aucun (GLSL sandbox)	modules signés Ed25519
Consentement	prompt pack vanilla	dialog in-game + clé serveur + perms
Révocation	n/a	CRL + kill switch service
Sandbox	GPU uniquement	permissions déclaratives (JVM sandbox = morte, JEP 486)
Zéro-clic compatible ?	oui (rien à installer)	non : le clic OS EST la barrière de sécurité — le conserver est un choix

Point clé §28 : le zéro-clic et la sécurité ne s'opposent pas dans V1 — parce que le clic unique n'installe pas du contenu mais une capacité générique (l'agent), et que tout contenu ultérieur reste gouverné par signatures/permissions/consentement in-game. Le clic OS n'arrive qu'une fois ; les centaines de « oui » in-game restent des choix de confiance par serveur, mémorisés.

3.7 Redémarrage : l'analyse précise (§27)

Besoin	Attach (M2) suffit ?	premain (M1) requis ?
HUD overlay	✓	—
Effets visuels custom (post, particles custom)	✓	—

Menus/dialogs custom enrichis	✓	—
Entités custom profondes (champs, IA enregistrée)	✗	✓
Keybinds enregistrés au boot	✗	✓
Transformations de classes de rendu	partiel (corps)	✓ complet

Le redémarrage n'est donc jamais une « étape d'installation » : c'est la bascule attach→premain, automatique au prochain lancement, invisible pour l'utilisateur.

Chapitre 4 — Faisabilité, PoC et ce qui reste à décider

4.1 Ce qui est prouvé vs ce qui est à construire

Brique	Statut	Preuve / effort
Plafond vanilla (V0)	Prouvé	Polymer 3,66 M ; catalogue rapport 1 ; 91 404 projets server-side
Agent premain + recettes	Prouvé	Fabric/Forge le font depuis 10 ans ; notre rapport 1
JDK_JAVA_OPTIONS injection	Prouvé (standard)	Doc Oracle vérifiée ; Datadog dd-trace en production massive
Attach à chaud	Prouvé (standard)	Attach API JVM ; IDE hot-reload ; JRebel
Version fantôme + libraries auto	Prouvé (standard)	Meta Fabric vérifié ce jour (inheritsFrom + 6 libs Maven)
Installeur Store full-trust	Faisable	_procs standard, revue ~2-4 sem
Service resident + détection JVM	Faisable	VirtualMachine.list() standard ; APM pattern
Plateforme serveur SDM	À construire	Le vrai chantier : gateway, Pack Studio, signatures, recettes par version
Handshake SDMP	À construire	Spéc rapport 1 + M6/M7 de celui-ci

4.2 PoC « Dream Flow » — 3 semaines, 4 jalons

J1 (semaine 1) — Le rêve visuel. Plugin serveur : au join d'un client vanilla, dialog deux boutons [**✱ Activer l'expérience**] [**Continuer sans**]. Non → V0 pur. Oui → cookie + (PoC : affichage d'instructions) — en production : Store. Livrable : la démo du flux de consentement, jouable.

J2 (semaine 1-2) — L'attach magique. Tool locale (pas encore Store) : `VirtualMachine.list()` → détection Minecraft → `loadAgent` → retransform `Gui.render` → HUD « SDM ACTIF » dessiné dans la session en cours, sur le serveur de J1, sans relancer. C'est la démonstration qui tue le débat : le joueur voit la barrière d'installation tomber en direct.

J3 (semaine 2) — L'env-var permanente. `setx JDK_JAVA_OPTIONS ...` + relance de l'instance → premain → plein pouvoir (recette d'un overlay custom + 1 entité virtuelle). Vérifier la survie à travers SKLauncher (présent sur la machine cible) et un second launcher.

J4 (semaine 3) — La version fantôme. Publier `sdm-26.2.json` + maven libraries + import Prism → instance auto-maintenue. Vérifier l'auto-update en bumpant une library.

Critère de succès global : un joueur lambda, depuis un client vanilla du Microsoft Store, rejoint le serveur, clique Oui + Obtenir, voit les effets dans sa session, et son prochain lancement est en plein pouvoir — sans jamais ouvrir un dossier ni comprendre un mot

technique.

4.3 Risques et mitigations

Risque	Gravité	Mitigation
Store refuse un tool full-trust	Moyen	Fallback : exe signé EV (1 clic « Exécuter ») + V2 launcher-tier
Antivirus heuristique sur l'attach	Moyen	Signature EV + transparence (open-source agent, page dédiée, hash publiés)
Mojang désapprouve officiellement	Faible-moyen	Position défendable : identique à Fabric (javagent), jamais de distribution d'assets, opt-out serveurs tiers ; ouvrir le dialogue tôt
Lunar/Badlion reset d'env	Faible	Couverts par le service attach (M2)
JDK_JAVA_OPTIONS retirée du JDK	Très faible	Standard documenté depuis JDK 9 ; JAVA_TOOL_OPTIONS (≥8) en secours — et V2 reste
Abus (serveur malveillant streamant un module fourbe)	Sérieux	Tout l'arsenal §28/§3.6 : signatures plateforme, perms, CRL, réputation, consentement par clé

4.4 Ce qui restera toujours vrai (honnêteté finale)

1. Le clic OS unique est le plancher physique — sept verrous, preuves classe-par-classe (chapitre 1). Aucune architecture honnête ne passe sous 1 clic une fois dans la vie. C'est aussi ce qui protège les joueurs : ce que nous voulons faire est exactement ce qu'un cheat voudrait faire.
2. La vérité reste serveur. Quelque soit le niveau, l'état de jeu autoritaire vit côté serveur ; le client exécute de la présentation.
3. Le coût des versions Minecraft est porté par la plateforme (recettes par version), pas par le joueur — c'est la promesse tenue du rapport 1, inchangée.
4. Le mode attaché est un sous-ensemble du premain. L'attach supprime le restart pour la majorité des effets, mais le plein pouvoir (champs, registres boot) exigera toujours le prochain lancement — transformé en bascule silencieuse.

4.5 Conclusion de la seconde investigation

La question finale : « existe-t-il un chemin vers un modding aussi puissant que Fabric/Forge/NeoForge et au-delà, avec zéro installation, zéro clic, ou au pire une action + restart ? »

Réponse : le zéro-clic strict est impossible (preuve exhaustivement établie), mais la plus petite concession est exactement : un clic « Obtenir » une fois dans la vie, zéro redémarrage obligatoire, zéro notion technique, tous launchers, toutes versions, tous

serveurs SDM pour toujours. L'architecture V3 (gradient V0+V1+V2) livre cette expérience par superposition : le vanilla pur reçoit 85 % du pouvoir ; le clic unique débloque les 98 % pour toujours avec activation immédiate en session (attach) et migration silencieuse au plein pouvoir (env-var) ; les launchers tiers héritent du canal de mise à jour natif (version fantôme).

Le rêve formulé — « t'arrives sur le serveur, on te propose oui/non, tu mets oui, et t'as tout » — est donc réalisé à un clic OS près, et ce clic est « Obtenir » dans le Microsoft Store : le geste le plus banal, le plus signé, le plus familier qui existe. En dessous, il n'y a plus que la triche.